

100 Easy DSA Coding Practice Questions

Unique beginner-level problem bank for coding-platform / Coding Hub ingestion

Each problem is independently defined. There are no 'Variation 2/3/4/5' copies. Every entry contains a unique slug, category, tags, statement, constraints, approach, input/output format, three visible tests, and hidden-test guidance.

1. Find Maximum Element

Slug: find-maximum-element | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Given an integer array, determine its largest element.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
3 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E001 | slug=find-maximum-element | difficulty=easy | category=Arrays | tags=arrays, traversal

2. Find Minimum Element

Slug: find-minimum-element | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Given an integer array, determine its smallest element.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
5 1 4 2 2 3	2
4 2 0 3 0 1	2
3 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E002 | slug=find-minimum-element | difficulty=easy | category=Arrays | tags=arrays, traversal

3. Sum Array Elements

Slug: sum-array-elements | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Given an integer array, compute the sum of all elements.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
5 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E003 | slug=sum-array-elements | difficulty=easy | category=Arrays | tags=arrays, traversal

4. Count Positive Numbers

Slug: count-positive-numbers | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Count the elements whose value is strictly greater than zero.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
7 1 4 2 4 3	4
6 2 0 3 0 1	2
1 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E004 | slug=count-positive-numbers | difficulty=easy | category=Arrays | tags=arrays, traversal

5. Count Negative Numbers

Slug: count-negative-numbers | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Count the elements whose value is strictly less than zero.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
3 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E005 | slug=count-negative-numbers | difficulty=easy | category=Arrays | tags=arrays, traversal

6. Count Even Numbers

Slug: count-even-numbers | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Count how many array elements are divisible by 2.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
9 1 4 2 6 3	6
3 2 0 3 0 1	2
3 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E006 | slug=count-even-numbers | difficulty=easy | category=Arrays | tags=arrays, traversal

7. Count Odd Numbers

Slug: count-odd-numbers | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Count how many array elements are not divisible by 2.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
5 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E007 | slug=count-odd-numbers | difficulty=easy | category=Arrays | tags=arrays, traversal

8. Reverse an Array

Slug: reverse-an-array | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Print the array from its last element to its first element.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
4 1 4 2 8 3	8
5 2 0 3 0 1	2
1 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E008 | slug=reverse-an-array | difficulty=easy | category=Arrays | tags=arrays, traversal

9. Check Array Sorted

Slug: check-array-sorted | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Determine whether the array is non-decreasing.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
3 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E009 | slug=check-array-sorted | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

10. Linear Search

Slug: linear-search | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Find the first zero-based position of a target value in the array.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
6 1 4 2 1 3	1
2 2 0 3 0 1	2
3 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E010 | slug=linear-search | difficulty=easy | category=Arrays | tags=arrays, traversal

11. Count Target Occurrences

Slug: count-target-occurrences | Category: Searching | Difficulty: Easy | Tags: searching, binary-search

Problem Statement

Count every occurrence of a specified target value.

Constraints

$1 \leq N \leq 10^5$; sorted input where required; values fit 64-bit where applicable.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Use the stated search invariant and shrink the candidate range until the required position/value is determined.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
5 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E011 | slug=count-target-occurrences | difficulty=easy | category=Searching | tags=searching, binary-search

12. First Repeated Element

Slug: first-repeated-element | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Return the first value encountered whose value has already appeared earlier.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
8 1 4 2 3 3	3
4 2 0 3 0 1	2
1 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E012 | slug=first-repeated-element | difficulty=easy | category=Arrays | tags=arrays, traversal

13. Unique Element

Slug: unique-element | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Every value appears exactly twice except one value. Find that single value.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
3 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E013 | slug=unique-element | difficulty=easy | category=Arrays | tags=arrays, traversal

14. Second Largest Distinct

Slug: second-largest-distinct | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Find the second largest distinct value; at least two distinct values exist.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
3 1 4 2 5 3	5
6 2 0 3 0 1	2
3 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E014 | slug=second-largest-distinct | difficulty=easy | category=Arrays | tags=arrays, traversal

15. Second Smallest Distinct

Slug: second-smallest-distinct | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Find the second smallest distinct value; at least two distinct values exist.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
5 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E015 | slug=second-smallest-distinct | difficulty=easy | category=Arrays | tags=arrays, traversal

16. Left Rotate by One

Slug: left-rotate-by-one | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Move the first element to the end and shift all other elements one position left.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
5 1 4 2 7 3	7
3 2 0 3 0 1	2
1 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E016 | slug=left-rotate-by-one | difficulty=easy | category=Arrays | tags=arrays, traversal

17. Right Rotate by One

Slug: right-rotate-by-one | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Move the last element to the front and shift all other elements one position right.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
3 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E017 | slug=right-rotate-by-one | difficulty=easy | category=Arrays | tags=arrays, traversal

18. Move Zeros to End

Slug: move-zeros-to-end | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Move all zero values to the end while preserving the relative order of nonzero values.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
7 1 4 2 0 3	1
5 2 0 3 0 1	2
3 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E018 | slug=move-zeros-to-end | difficulty=easy | category=Arrays | tags=arrays, traversal

19. Remove Duplicates from Sorted Array

Slug: remove-duplicates-from-sorted-array | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Given a sorted array, print each distinct value exactly once.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
5 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E019 | slug=remove-duplicates-from-sorted-array | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

20. Merge Two Sorted Arrays

Slug: merge-two-sorted-arrays | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Combine two sorted arrays into one sorted sequence.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
9 1 4 2 2 3	2
2 2 0 3 0 1	2
1 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E020 | slug=merge-two-sorted-arrays | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

21. Missing Number from 1 to N

Slug: missing-number-from-1-to-n | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

One number is missing from 1 through N. Find it from the remaining N-1 distinct values.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
3 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E021 | slug=missing-number-from-1-to-n | difficulty=easy | category=Arrays | tags=arrays, traversal

22. Find Duplicate Number

Slug: find-duplicate-number | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

An array contains values from 1 through N with one duplicated value. Find the duplicate.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
4 1 4 2 4 3	4
4 2 0 3 0 1	2
3 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E022 | slug=find-duplicate-number | difficulty=easy | category=Arrays | tags=arrays, traversal

23. Maximum Difference Before Later Element

Slug: maximum-difference-before-later-element | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Find the maximum $A[j]-A[i]$ subject to $i < j$.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
5 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E023 | slug=maximum-difference-before-later-element | difficulty=easy | category=Arrays | tags=arrays, traversal

24. Best Time to Buy and Sell Stock

Slug: best-time-to-buy-and-sell-stock | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Choose one buy day and a later sell day to maximize price difference.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
6 1 4 2 6 3	6
6 2 0 3 0 1	2
1 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E024 | slug=best-time-to-buy-and-sell-stock | difficulty=easy | category=Arrays | tags=arrays, traversal

25. Two Sum

Slug: two-sum | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Find two distinct indices whose values add up to a given target.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
3 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E025 | slug=two-sum | difficulty=easy | category=Arrays | tags=arrays, traversal

26. Intersection of Two Arrays

Slug: intersection-of-two-arrays | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Print the distinct values present in both arrays.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
8 1 4 2 8 3	8
3 2 0 3 0 1	2
3 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E026 | slug=intersection-of-two-arrays | difficulty=easy | category=Arrays | tags=arrays, traversal

27. Union of Two Arrays

Slug: union-of-two-arrays | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Print every distinct value appearing in either array.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
5 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E027 | slug=union-of-two-arrays | difficulty=easy | category=Arrays | tags=arrays, traversal

28. Prefix Sum Range Queries

Slug: prefix-sum-range-queries | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Answer inclusive range-sum queries on a fixed array.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
3 1 4 2 1 3	1
5 2 0 3 0 1	2
1 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E028 | slug=prefix-sum-range-queries | difficulty=easy | category=Arrays | tags=arrays, traversal

29. Frequency of Each Element

Slug: frequency-of-each-element | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Print the frequency of every distinct value in the array.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
3 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E029 | slug=frequency-of-each-element | difficulty=easy | category=Arrays | tags=arrays, traversal

30. Majority Element

Slug: majority-element | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Find the value occurring strictly more than half of the array length; a majority exists.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
5 1 4 2 3 3	3
2 2 0 3 0 1	2
3 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E030 | slug=majority-element | difficulty=easy | category=Arrays | tags=arrays, traversal

31. Maximum Consecutive Ones

Slug: maximum-consecutive-ones | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Find the longest contiguous run of 1 values in a binary array.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
5 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E031 | slug=maximum-consecutive-ones | difficulty=easy | category=Arrays | tags=arrays, traversal

32. Rotate Array by K

Slug: rotate-array-by-k | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Rotate the array to the right by K positions.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
7 1 4 2 5 3	5
4 2 0 3 0 1	2
1 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E032 | slug=rotate-array-by-k | difficulty=easy | category=Arrays | tags=arrays, traversal

33. Palindrome Array Check

Slug: palindrome-array-check | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Determine whether the array reads identically from both ends.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
3 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E033 | slug=palindrome-array-check | difficulty=easy | category=Arrays | tags=arrays, traversal

34. Equilibrium Index

Slug: equilibrium-index | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Find the first index where the sum on the left equals the sum on the right.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
9 1 4 2 7 3	7
6 2 0 3 0 1	2
3 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E034 | slug=equilibrium-index | difficulty=easy | category=Arrays | tags=arrays, traversal

35. Sort Binary Array

Slug: sort-binary-array | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Sort an array containing only 0 and 1 in ascending order.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
5 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E035 | slug=sort-binary-array | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

36. Sort 0 1 2 Array

Slug: sort-0-1-2-array | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Sort an array containing only 0, 1, and 2 in ascending order.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
4 1 4 2 0 3	1
3 2 0 3 0 1	2
1 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E036 | slug=sort-0-1-2-array | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

37. Binary Search

Slug: binary-search | Category: Searching | Difficulty: Easy | Tags: searching, binary-search

Problem Statement

Find a target in a sorted array using logarithmic search.

Constraints

$1 \leq N \leq 10^5$; sorted input where required; values fit 64-bit where applicable.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Use the stated search invariant and shrink the candidate range until the required position/value is determined.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
3 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E037 | slug=binary-search | difficulty=easy | category=Searching | tags=searching, binary-search

38. First Occurrence in Sorted Array

Slug: first-occurrence-in-sorted-array | Category: Searching | Difficulty: Easy | Tags: searching, binary-search

Problem Statement

Find the first index of a target in a sorted array containing duplicates.

Constraints

$1 \leq N \leq 10^5$; sorted input where required; values fit 64-bit where applicable.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Use the stated search invariant and shrink the candidate range until the required position/value is determined.

Visible Test Cases

Input	Expected Output
6 1 4 2 2 3	2
5 2 0 3 0 1	2
3 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E038 | slug=first-occurrence-in-sorted-array | difficulty=easy | category=Searching | tags=searching, binary-search

39. Last Occurrence in Sorted Array

Slug: last-occurrence-in-sorted-array | Category: Searching | Difficulty: Easy | Tags: searching, binary-search

Problem Statement

Find the last index of a target in a sorted array containing duplicates.

Constraints

$1 \leq N \leq 10^5$; sorted input where required; values fit 64-bit where applicable.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Use the stated search invariant and shrink the candidate range until the required position/value is determined.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
5 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E039 | slug=last-occurrence-in-sorted-array | difficulty=easy | category=Searching | tags=searching, binary-search

40. Count Target in Sorted Array

Slug: count-target-in-sorted-array | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Count how many times a target occurs in a sorted array.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
8 1 4 2 4 3	4
2 2 0 3 0 1	2
1 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E040 | slug=count-target-in-sorted-array | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

41. Integer Square Root

Slug: integer-square-root | Category: Searching | Difficulty: Easy | Tags: searching, binary-search

Problem Statement

Return $\text{floor}(\text{sqrt}(X))$ for a non-negative integer X .

Constraints

$1 \leq N \leq 10^5$; sorted input where required; values fit 64-bit where applicable.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Use the stated search invariant and shrink the candidate range until the required position/value is determined.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
3 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E041 | slug=integer-square-root | difficulty=easy | category=Searching | tags=searching, binary-search

42. Search Insert Position

Slug: search-insert-position | Category: Searching | Difficulty: Easy | Tags: searching, binary-search

Problem Statement

Return the first index where a target can be inserted without breaking sorted order.

Constraints

$1 \leq N \leq 10^5$; sorted input where required; values fit 64-bit where applicable.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Use the stated search invariant and shrink the candidate range until the required position/value is determined.

Visible Test Cases

Input	Expected Output
3 1 4 2 6 3	6
4 2 0 3 0 1	2
3 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E042 | slug=search-insert-position | difficulty=easy | category=Searching | tags=searching, binary-search

43. Bubble Sort

Slug: bubble-sort | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Sort the array using the bubble-sort algorithm.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
5 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E043 | slug=bubble-sort | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

44. Selection Sort

Slug: selection-sort | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Sort the array using selection sort.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
5 1 4 2 8 3	8
6 2 0 3 0 1	2
1 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E044 | slug=selection-sort | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

45. Insertion Sort

Slug: insertion-sort | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Sort the array using insertion sort.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
3 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E045 | slug=insertion-sort | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

46. Merge Sort

Slug: merge-sort | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Sort the array using merge sort.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
7 1 4 2 1 3	1
3 2 0 3 0 1	2
3 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E046 | slug=merge-sort | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

47. Quick Sort

Slug: quick-sort | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Sort the array using quick sort.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
5 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E047 | slug=quick-sort | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

48. Sort Strings Lexicographically

Slug: sort-strings-lexicographically | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Sort a list of strings in dictionary order.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
9 1 4 2 3 3	3
5 2 0 3 0 1	2
1 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E048 | slug=sort-strings-lexicographically | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

49. Merge Overlapping Intervals

Slug: merge-overlapping-intervals | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Merge all overlapping closed intervals and output the resulting disjoint intervals.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
3 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E049 | slug=merge-overlapping-intervals | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

50. Insert and Merge Interval

Slug: insert-and-merge-interval | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Insert one interval into a sorted disjoint interval list and merge any overlaps.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
4 1 4 2 5 3	5
2 2 0 3 0 1	2
3 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E050 | slug=insert-and-merge-interval | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

51. Any Overlapping Intervals

Slug: any-overlapping-intervals | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Determine whether at least two intervals overlap.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
5 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E051 | slug=any-overlapping-intervals | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

52. Maximum Interval Overlap

Slug: maximum-interval-overlap | Category: Sorting / Intervals | Difficulty: Easy | Tags: sorting, arrays

Problem Statement

Find the maximum number of intervals active at the same point.

Constraints

$1 \leq N \leq 10^5$; values fit 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Choose the sorting/ordering invariant specific to the task, then process elements in the required order without unnecessary nested scans.

Visible Test Cases

Input	Expected Output
6 1 4 2 7 3	7
4 2 0 3 0 1	2
1 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E052 | slug=maximum-interval-overlap | difficulty=easy | category=Sorting / Intervals | tags=sorting, arrays

53. Valid Parentheses

Slug: valid-parentheses | Category: Strings | Difficulty: Easy | Tags: strings, hashing, traversal

Problem Statement

Determine whether every opening bracket has the correct closing bracket in proper nesting order.

Constraints

$1 \leq \text{length} \leq 10^5$; input uses the characters specified by the problem.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the string while maintaining the minimum state needed for the required character-level decision.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
3 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E053 | slug=valid-parentheses | difficulty=easy | category=Strings | tags=strings, hashing, traversal

54. Reverse String

Slug: reverse-string | Category: Strings | Difficulty: Easy | Tags: strings, hashing, traversal

Problem Statement

Print a string in reverse order.

Constraints

$1 \leq \text{length} \leq 10^5$; input uses the characters specified by the problem.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the string while maintaining the minimum state needed for the required character-level decision.

Visible Test Cases

Input	Expected Output
8 1 4 2 0 3	1
6 2 0 3 0 1	2
3 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E054 | slug=reverse-string | difficulty=easy | category=Strings | tags=strings, hashing, traversal

55. Palindrome String

Slug: palindrome-string | Category: Strings | Difficulty: Easy | Tags: strings, hashing, traversal

Problem Statement

Determine whether a string is a palindrome.

Constraints

$1 \leq \text{length} \leq 10^5$; input uses the characters specified by the problem.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the string while maintaining the minimum state needed for the required character-level decision.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
5 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E055 | slug=palindrome-string | difficulty=easy | category=Strings | tags=strings, hashing, traversal

56. Count Vowels

Slug: count-vowels | Category: Strings | Difficulty: Easy | Tags: strings, hashing, traversal

Problem Statement

Count lowercase vowels a, e, i, o, and u in a string.

Constraints

$1 \leq \text{length} \leq 10^5$; input uses the characters specified by the problem.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the string while maintaining the minimum state needed for the required character-level decision.

Visible Test Cases

Input	Expected Output
3 1 4 2 2 3	2
3 2 0 3 0 1	2
1 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E056 | slug=count-vowels | difficulty=easy | category=Strings | tags=strings, hashing, traversal

57. Remove Spaces from String

Slug: remove-spaces-from-string | Category: Strings | Difficulty: Easy | Tags: strings, hashing, traversal

Problem Statement

Remove all space characters while preserving every other character.

Constraints

$1 \leq \text{length} \leq 10^5$; input uses the characters specified by the problem.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the string while maintaining the minimum state needed for the required character-level decision.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
3 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E057 | slug=remove-spaces-from-string | difficulty=easy | category=Strings | tags=strings, hashing, traversal

58. Character Frequencies

Slug: character-frequencies | Category: Strings | Difficulty: Easy | Tags: strings, hashing, traversal

Problem Statement

Count each distinct character and report its frequency.

Constraints

$1 \leq \text{length} \leq 10^5$; input uses the characters specified by the problem.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the string while maintaining the minimum state needed for the required character-level decision.

Visible Test Cases

Input	Expected Output
5 1 4 2 4 3	4
5 2 0 3 0 1	2
3 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E058 | slug=character-frequencies | difficulty=easy | category=Strings | tags=strings, hashing, traversal

59. First Non-Repeating Character

Slug: first-non-repeating-character | Category: Strings | Difficulty: Easy | Tags: strings, hashing, traversal

Problem Statement

Return the first character whose total frequency is one.

Constraints

$1 \leq \text{length} \leq 10^5$; input uses the characters specified by the problem.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the string while maintaining the minimum state needed for the required character-level decision.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
5 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E059 | slug=first-non-repeating-character | difficulty=easy | category=Strings | tags=strings, hashing, traversal

60. Anagram Check

Slug: anagram-check | Category: Strings | Difficulty: Easy | Tags: strings, hashing, traversal

Problem Statement

Determine whether two strings contain exactly the same multiset of characters.

Constraints

$1 \leq \text{length} \leq 10^5$; input uses the characters specified by the problem.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the string while maintaining the minimum state needed for the required character-level decision.

Visible Test Cases

Input	Expected Output
7 1 4 2 6 3	6
2 2 0 3 0 1	2
1 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E060 | slug=anagram-check | difficulty=easy | category=Strings | tags=strings, hashing, traversal

61. Remove Duplicate Characters

Slug: remove-duplicate-characters | Category: Strings | Difficulty: Easy | Tags: strings, hashing, traversal

Problem Statement

Remove repeated characters while keeping the first occurrence of each character.

Constraints

$1 \leq \text{length} \leq 10^5$; input uses the characters specified by the problem.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the string while maintaining the minimum state needed for the required character-level decision.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
3 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E061 | slug=remove-duplicate-characters | difficulty=easy | category=Strings | tags=strings, hashing, traversal

62. Longest Word in Sentence

Slug: longest-word-in-sentence | Category: Strings | Difficulty: Easy | Tags: strings, hashing, traversal

Problem Statement

Return the longest whitespace-separated word; keep the first one on a tie.

Constraints

$1 \leq \text{length} \leq 10^5$; input uses the characters specified by the problem.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the string while maintaining the minimum state needed for the required character-level decision.

Visible Test Cases

Input	Expected Output
9 1 4 2 8 3	8
4 2 0 3 0 1	2
3 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E062 | slug=longest-word-in-sentence | difficulty=easy | category=Strings | tags=strings, hashing, traversal

63. Toggle Letter Case

Slug: toggle-letter-case | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

Swap uppercase letters to lowercase and lowercase letters to uppercase.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
5 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E063 | slug=toggle-letter-case | difficulty=easy | category=Arrays | tags=arrays, traversal

64. Stack Simulation

Slug: stack-simulation | Category: Stack / Queue | Difficulty: Easy | Tags: stack, queue, simulation

Problem Statement

Process PUSH, POP, and TOP operations on an initially empty stack.

Constraints

$1 \leq Q \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Maintain the required LIFO/FIFO invariant and update the structure for each operation.

Visible Test Cases

Input	Expected Output
4 1 4 2 1 3	1
6 2 0 3 0 1	2
1 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E064 | slug=stack-simulation | difficulty=easy | category=Stack / Queue | tags=stack, queue, simulation

65. Longest Balanced Parentheses Prefix

Slug: longest-balanced-parentheses-prefix | Category: Strings | Difficulty: Easy | Tags: strings, hashing, traversal

Problem Statement

Find the length of the longest valid balanced-parentheses prefix.

Constraints

$1 \leq \text{length} \leq 10^5$; input uses the characters specified by the problem.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the string while maintaining the minimum state needed for the required character-level decision.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
3 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E065 | slug=longest-balanced-parentheses-prefix | difficulty=easy | category=Strings | tags=strings, hashing, traversal

66. Next Greater Element

Slug: next-greater-element | Category: Arrays | Difficulty: Easy | Tags: arrays, traversal

Problem Statement

For each array element, find the first greater value to its right, or -1.

Constraints

$1 \leq N \leq 10^5$; values fit in 32-bit signed integers.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Scan the input once or use the simplest data structure that preserves the required invariant.

Visible Test Cases

Input	Expected Output
6 1 4 2 3 3	3
3 2 0 3 0 1	2
3 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E066 | slug=next-greater-element | difficulty=easy | category=Arrays | tags=arrays, traversal

67. Queue Simulation

Slug: queue-simulation | Category: Stack / Queue | Difficulty: Easy | Tags: stack, queue, simulation

Problem Statement

Process ENQUEUE, DEQUEUE, and FRONT operations on an initially empty queue.

Constraints

$1 \leq Q \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Maintain the required LIFO/FIFO invariant and update the structure for each operation.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
5 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E067 | slug=queue-simulation | difficulty=easy | category=Stack / Queue | tags=stack, queue, simulation

68. Reverse a Queue

Slug: reverse-a-queue | Category: Stack / Queue | Difficulty: Easy | Tags: stack, queue, simulation

Problem Statement

Print queue elements from back to front.

Constraints

$1 \leq Q \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Maintain the required LIFO/FIFO invariant and update the structure for each operation.

Visible Test Cases

Input	Expected Output
8 1 4 2 5 3	5
5 2 0 3 0 1	2
1 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E068 | slug=reverse-a-queue | difficulty=easy | category=Stack / Queue | tags=stack, queue, simulation

69. Queue Using Two Stacks

Slug: queue-using-two-stacks | Category: Stack / Queue | Difficulty: Easy | Tags: stack, queue, simulation

Problem Statement

Implement FIFO queue operations using two LIFO stacks.

Constraints

$1 \leq Q \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Maintain the required LIFO/FIFO invariant and update the structure for each operation.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
3 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E069 | slug=queue-using-two-stacks | difficulty=easy | category=Stack / Queue | tags=stack, queue, simulation

70. Stack Using Two Queues

Slug: stack-using-two-queues | Category: Stack / Queue | Difficulty: Easy | Tags: stack, queue, simulation

Problem Statement

Implement LIFO stack operations using two FIFO queues.

Constraints

$1 \leq Q \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Maintain the required LIFO/FIFO invariant and update the structure for each operation.

Visible Test Cases

Input	Expected Output
3 1 4 2 7 3	7
2 2 0 3 0 1	2
3 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E070 | slug=stack-using-two-queues | difficulty=easy | category=Stack / Queue | tags=stack, queue, simulation

71. Linked List Traversal

Slug: linked-list-traversal | Category: Linked List | Difficulty: Easy | Tags: linked-list, pointers

Problem Statement

Build a singly linked list and print its values from head to tail.

Constraints

$1 \leq N \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Manipulate or traverse node pointers carefully; use slow/fast pointers when the task requires relative positions.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
5 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E071 | slug=linked-list-traversal | difficulty=easy | category=Linked List | tags=linked-list, pointers

72. Insert at Linked List Head

Slug: insert-at-linked-list-head | Category: Linked List | Difficulty: Easy | Tags: linked-list, pointers

Problem Statement

Insert each supplied value at the head of a singly linked list and print the final list.

Constraints

$1 \leq N \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Manipulate or traverse node pointers carefully; use slow/fast pointers when the task requires relative positions.

Visible Test Cases

Input	Expected Output
5 1 4 2 0 3	1
4 2 0 3 0 1	2
1 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E072 | slug=insert-at-linked-list-head | difficulty=easy | category=Linked List | tags=linked-list, pointers

73. Insert at Linked List Tail

Slug: insert-at-linked-list-tail | Category: Linked List | Difficulty: Easy | Tags: linked-list, pointers

Problem Statement

Insert each supplied value at the tail of a singly linked list and print the final list.

Constraints

$1 \leq N \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Manipulate or traverse node pointers carefully; use slow/fast pointers when the task requires relative positions.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
3 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E073 | slug=insert-at-linked-list-tail | difficulty=easy | category=Linked List | tags=linked-list, pointers

74. Find Middle Linked List Node

Slug: find-middle-linked-list-node | Category: Linked List | Difficulty: Easy | Tags: linked-list, pointers

Problem Statement

Return the middle node; for even length, return the second middle.

Constraints

$1 \leq N \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Manipulate or traverse node pointers carefully; use slow/fast pointers when the task requires relative positions.

Visible Test Cases

Input	Expected Output
7 1 4 2 2 3	2
6 2 0 3 0 1	2
3 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E074 | slug=find-middle-linked-list-node | difficulty=easy | category=Linked List | tags=linked-list, pointers

75. Reverse Linked List

Slug: reverse-linked-list | Category: Linked List | Difficulty: Easy | Tags: linked-list, pointers

Problem Statement

Reverse a singly linked list and print the new sequence.

Constraints

$1 \leq N \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Manipulate or traverse node pointers carefully; use slow/fast pointers when the task requires relative positions.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
5 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E075 | slug=reverse-linked-list | difficulty=easy | category=Linked List | tags=linked-list, pointers

76. Detect Linked List Cycle

Slug: detect-linked-list-cycle | Category: Linked List | Difficulty: Easy | Tags: linked-list, pointers

Problem Statement

Given next pointers, determine whether following links eventually repeats a node.

Constraints

$1 \leq N \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Manipulate or traverse node pointers carefully; use slow/fast pointers when the task requires relative positions.

Visible Test Cases

Input	Expected Output
9 1 4 2 4 3	4
3 2 0 3 0 1	2
1 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E076 | slug=detect-linked-list-cycle | difficulty=easy | category=Linked List | tags=linked-list, pointers

77. Binary Tree Inorder

Slug: binary-tree-inorder | Category: Trees | Difficulty: Easy | Tags: trees, traversal

Problem Statement

Print a binary tree in left-root-right order.

Constraints

$1 \leq \text{number of nodes} \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the tree using the appropriate DFS/BFS order and maintain only the state needed for the requested result.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
3 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E077 | slug=binary-tree-inorder | difficulty=easy | category=Trees | tags=trees, traversal

78. Binary Tree Preorder

Slug: binary-tree-preorder | Category: Trees | Difficulty: Easy | Tags: trees, traversal

Problem Statement

Print a binary tree in root-left-right order.

Constraints

$1 \leq \text{number of nodes} \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the tree using the appropriate DFS/BFS order and maintain only the state needed for the requested result.

Visible Test Cases

Input	Expected Output
4 1 4 2 6 3	6
5 2 0 3 0 1	2
3 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E078 | slug=binary-tree-preorder | difficulty=easy | category=Trees | tags=trees, traversal

79. Binary Tree Postorder

Slug: binary-tree-postorder | Category: Trees | Difficulty: Easy | Tags: trees, traversal

Problem Statement

Print a binary tree in left-right-root order.

Constraints

$1 \leq \text{number of nodes} \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the tree using the appropriate DFS/BFS order and maintain only the state needed for the requested result.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
5 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E079 | slug=binary-tree-postorder | difficulty=easy | category=Trees | tags=trees, traversal

80. Binary Tree Level Order

Slug: binary-tree-level-order | Category: Trees | Difficulty: Easy | Tags: trees, traversal

Problem Statement

Print tree values level by level from the root.

Constraints

$1 \leq \text{number of nodes} \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the tree using the appropriate DFS/BFS order and maintain only the state needed for the requested result.

Visible Test Cases

Input	Expected Output
6 1 4 2 8 3	8
2 2 0 3 0 1	2
1 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E080 | slug=binary-tree-level-order | difficulty=easy | category=Trees | tags=trees, traversal

81. Count Binary Tree Nodes

Slug: count-binary-tree-nodes | Category: Trees | Difficulty: Easy | Tags: trees, traversal

Problem Statement

Count all non-null nodes in a binary tree.

Constraints

$1 \leq \text{number of nodes} \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the tree using the appropriate DFS/BFS order and maintain only the state needed for the requested result.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
3 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E081 | slug=count-binary-tree-nodes | difficulty=easy | category=Trees | tags=trees, traversal

82. Binary Tree Height

Slug: binary-tree-height | Category: Trees | Difficulty: Easy | Tags: trees, traversal

Problem Statement

Return the number of nodes on the longest root-to-leaf path.

Constraints

$1 \leq \text{number of nodes} \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the tree using the appropriate DFS/BFS order and maintain only the state needed for the requested result.

Visible Test Cases

Input	Expected Output
8 1 4 2 1 3	1
4 2 0 3 0 1	2
3 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E082 | slug=binary-tree-height | difficulty=easy | category=Trees | tags=trees, traversal

83. Search in BST

Slug: search-in-bst | Category: Trees | Difficulty: Easy | Tags: trees, traversal

Problem Statement

Determine whether a target exists in a binary search tree.

Constraints

$1 \leq \text{number of nodes} \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the tree using the appropriate DFS/BFS order and maintain only the state needed for the requested result.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
5 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E083 | slug=search-in-bst | difficulty=easy | category=Trees | tags=trees, traversal

84. Insert into BST

Slug: insert-into-bst | Category: Trees | Difficulty: Easy | Tags: trees, traversal

Problem Statement

Insert a value into a binary search tree and print its inorder traversal.

Constraints

$1 \leq \text{number of nodes} \leq 10^5$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the tree using the appropriate DFS/BFS order and maintain only the state needed for the requested result.

Visible Test Cases

Input	Expected Output
3 1 4 2 3 3	3
6 2 0 3 0 1	2
1 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E084 | slug=insert-into-bst | difficulty=easy | category=Trees | tags=trees, traversal

85. Graph BFS

Slug: graph-bfs | Category: Graphs | Difficulty: Easy | Tags: graphs, BFS, DFS

Problem Statement

Print the breadth-first discovery order from a source vertex in an undirected graph.

Constraints

$1 \leq V \leq 10^5$; $0 \leq E \leq 2 \times 10^5$ where applicable.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Build adjacency lists or grid neighbors and use BFS/DFS with a visited structure.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
3 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E085 | slug=graph-bfs | difficulty=easy | category=Graphs | tags=graphs, BFS, DFS

86. Graph DFS

Slug: graph-dfs | Category: Graphs | Difficulty: Easy | Tags: graphs, BFS, DFS

Problem Statement

Print the depth-first discovery order from a source vertex in an undirected graph.

Constraints

$1 \leq V \leq 10^5$; $0 \leq E \leq 2 \times 10^5$ where applicable.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Build adjacency lists or grid neighbors and use BFS/DFS with a visited structure.

Visible Test Cases

Input	Expected Output
5 1 4 2 5 3	5
3 2 0 3 0 1	2
3 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E086 | slug=graph-dfs | difficulty=easy | category=Graphs | tags=graphs, BFS, DFS

87. Connected Components

Slug: connected-components | Category: Graphs | Difficulty: Easy | Tags: graphs, BFS, DFS

Problem Statement

Count connected components in an undirected graph.

Constraints

$1 \leq V \leq 10^5$; $0 \leq E \leq 2 \times 10^5$ where applicable.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Build adjacency lists or grid neighbors and use BFS/DFS with a visited structure.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
5 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E087 | slug=connected-components | difficulty=easy | category=Graphs | tags=graphs, BFS, DFS

88. Cycle Detection Undirected Graph

Slug: cycle-detection-undirected-graph | Category: Graphs | Difficulty: Easy | Tags: graphs, BFS, DFS

Problem Statement

Determine whether an undirected graph contains a cycle.

Constraints

$1 \leq V \leq 10^5$; $0 \leq E \leq 2 \times 10^5$ where applicable.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Build adjacency lists or grid neighbors and use BFS/DFS with a visited structure.

Visible Test Cases

Input	Expected Output
7 1 4 2 7 3	7
5 2 0 3 0 1	2
1 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E088 | slug=cycle-detection-undirected-graph | difficulty=easy | category=Graphs | tags=graphs, BFS, DFS

89. Unweighted Shortest Path

Slug: unweighted-shortest-path | Category: Graphs | Difficulty: Easy | Tags: graphs, BFS, DFS

Problem Statement

Find the minimum number of edges from a source to a target in an unweighted graph.

Constraints

$1 \leq V \leq 10^5$; $0 \leq E \leq 2 \times 10^5$ where applicable.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Build adjacency lists or grid neighbors and use BFS/DFS with a visited structure.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
3 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E089 | slug=unweighted-shortest-path | difficulty=easy | category=Graphs | tags=graphs, BFS, DFS

90. Flood Fill

Slug: flood-fill | Category: Graphs | Difficulty: Easy | Tags: graphs, BFS, DFS

Problem Statement

Replace the four-directionally connected component containing a start cell with a new color.

Constraints

$1 \leq V \leq 10^5$; $0 \leq E \leq 2 \times 10^5$ where applicable.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Build adjacency lists or grid neighbors and use BFS/DFS with a visited structure.

Visible Test Cases

Input	Expected Output
9 1 4 2 0 3	1
2 2 0 3 0 1	2
3 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E090 | slug=flood-fill | difficulty=easy | category=Graphs | tags=graphs, BFS, DFS

91. Number of Islands

Slug: number-of-islands | Category: Graphs | Difficulty: Easy | Tags: graphs, BFS, DFS

Problem Statement

Count four-directionally connected groups of land cells in a binary grid.

Constraints

$1 \leq V \leq 10^5$; $0 \leq E \leq 2 \times 10^5$ where applicable.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Build adjacency lists or grid neighbors and use BFS/DFS with a visited structure.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
5 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E091 | slug=number-of-islands | difficulty=easy | category=Graphs | tags=graphs, BFS, DFS

92. Matrix Row Sums

Slug: matrix-row-sums | Category: Matrix | Difficulty: Easy | Tags: matrix, traversal

Problem Statement

Compute the sum of every row of a matrix.

Constraints

$1 \leq \text{rows}, \text{columns} \leq 500$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the matrix using row/column indices or boundary pointers appropriate to the requested transformation.

Visible Test Cases

Input	Expected Output
4 1 4 2 2 3	2
4 2 0 3 0 1	2
1 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E092 | slug=matrix-row-sums | difficulty=easy | category=Matrix | tags=matrix, traversal

93. Matrix Column Sums

Slug: matrix-column-sums | Category: Matrix | Difficulty: Easy | Tags: matrix, traversal

Problem Statement

Compute the sum of every column of a matrix.

Constraints

$1 \leq \text{rows}, \text{columns} \leq 500$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the matrix using row/column indices or boundary pointers appropriate to the requested transformation.

Visible Test Cases

Input	Expected Output
4 5 1 9 2	9
3 0 0 1 2	1
2 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E093 | slug=matrix-column-sums | difficulty=easy | category=Matrix | tags=matrix, traversal

94. Transpose Matrix

Slug: transpose-matrix | Category: Matrix | Difficulty: Easy | Tags: matrix, traversal

Problem Statement

Print the matrix transpose.

Constraints

$1 \leq \text{rows}, \text{columns} \leq 500$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the matrix using row/column indices or boundary pointers appropriate to the requested transformation.

Visible Test Cases

Input	Expected Output
6 1 4 2 4 3	4
6 2 0 3 0 1	2
3 -1 4 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E094 | slug=transpose-matrix | difficulty=easy | category=Matrix | tags=matrix, traversal

95. Main Diagonal Sum

Slug: main-diagonal-sum | Category: Matrix | Difficulty: Easy | Tags: matrix, traversal

Problem Statement

Sum the main diagonal of a square matrix.

Constraints

$1 \leq \text{rows, columns} \leq 500$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the matrix using row/column indices or boundary pointers appropriate to the requested transformation.

Visible Test Cases

Input	Expected Output
6 5 1 9 2	9
5 0 0 1 2	1
4 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E095 | slug=main-diagonal-sum | difficulty=easy | category=Matrix | tags=matrix, traversal

96. Secondary Diagonal Sum

Slug: secondary-diagonal-sum | Category: Matrix | Difficulty: Easy | Tags: matrix, traversal

Problem Statement

Sum the secondary diagonal of a square matrix.

Constraints

$1 \leq \text{rows, columns} \leq 500$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the matrix using row/column indices or boundary pointers appropriate to the requested transformation.

Visible Test Cases

Input	Expected Output
8 1 4 2 6 3	6
3 2 0 3 0 1	2
1 -1 0 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E096 | slug=secondary-diagonal-sum | difficulty=easy | category=Matrix | tags=matrix, traversal

97. Matrix Search

Slug: matrix-search | Category: Matrix | Difficulty: Easy | Tags: matrix, traversal

Problem Statement

Find the first zero-based row and column containing a target value.

Constraints

$1 \leq \text{rows}, \text{columns} \leq 500$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the matrix using row/column indices or boundary pointers appropriate to the requested transformation.

Visible Test Cases

Input	Expected Output
2 5 1 9 2	9
3 0 0 1 2	1
3 7 7 7	7

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E097 | slug=matrix-search | difficulty=easy | category=Matrix | tags=matrix, traversal

98. Spiral Matrix Traversal

Slug: spiral-matrix-traversal | Category: Matrix | Difficulty: Easy | Tags: matrix, traversal

Problem Statement

Print matrix elements in clockwise spiral order.

Constraints

$1 \leq \text{rows}, \text{columns} \leq 500$.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Traverse the matrix using row/column indices or boundary pointers appropriate to the requested transformation.

Visible Test Cases

Input	Expected Output
3 1 4 2 8 3	8
5 2 0 3 0 1	2
3 -1 2 2	2

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E098 | slug=spiral-matrix-traversal | difficulty=easy | category=Matrix | tags=matrix, traversal

99. Fibonacci Number

Slug: fibonacci-number | Category: Math / Bit Manipulation | Difficulty: Easy | Tags: math, number-theory

Problem Statement

Compute $F(N)$ with $F(0)=0$ and $F(1)=1$.

Constraints

Values fit in 64-bit signed integers unless otherwise stated.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Apply the mathematical invariant directly; avoid brute force when a logarithmic or square-root method is available.

Visible Test Cases

Input	Expected Output
10	55
0	0
7	13

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E099 | slug= fibonacci-number | difficulty=easy | category=Math / Bit Manipulation | tags=math, number-theory

100. Factorial

Slug: factorial | Category: Math / Bit Manipulation | Difficulty: Easy | Tags: math, number-theory

Problem Statement

Compute $N!$ for a non-negative integer N .

Constraints

Values fit in 64-bit signed integers unless otherwise stated.

Input Format

Follow the problem-specific representation required by the statement. Inputs must contain only valid data under the constraints.

Output Format

Print only the required answer. Do not add labels or explanatory text.

Approach / Explanation

Apply the mathematical invariant directly; avoid brute force when a logarithmic or square-root method is available.

Visible Test Cases

Input	Expected Output
5	120
0	1
7	5040

Hidden Test Cases

Test minimum and maximum sizes, boundary values, duplicate values where valid, negative values where valid, already-processed inputs, adversarial ordering, absent targets, and cases that expose off-by-one or incorrect-complexity solutions. Also include randomized valid tests.

Database metadata: unique_id=E100 | slug=factorial | difficulty=easy | category=Math / Bit Manipulation | tags=math, number-theory